

# TP2 — Sistemas Distribuídos

## Serviços de Análise e Monitorização Urbana para One Health

### Roadmap de Desenvolvimento

(com justificação de decisões arquiteturais)

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| Disciplina      | Sistemas Distribuídos — UTAD   ECT   DE 2025/2026       |
| Docentes        | Hugo Paredes   Tiago Pinto   Cristiano Pendão           |
| Data limite     | 29 de maio de 2026 (apresentação na PL seguinte)        |
| Stack escolhida | .NET 8/9 (C#) + Python 3.11 + RabbitMQ + MongoDB + gRPC |
| Entrega         | Moodle — relatório (4 págs) + código (GitHub/GitLab)    |

## Nota sobre os critérios de avaliação

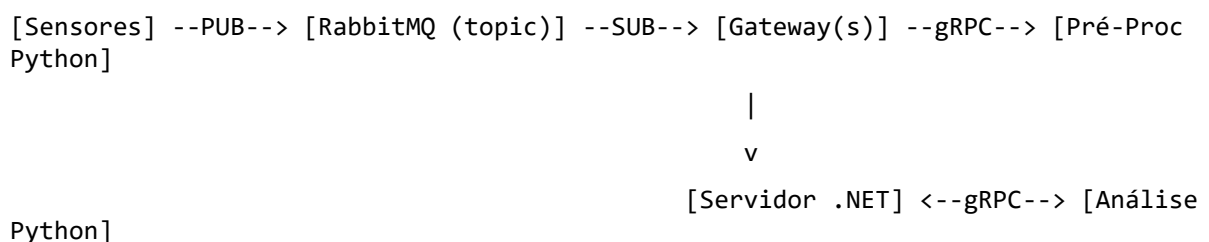
Após conversa com o docente, fica claro que um dos critérios principais de avaliação é a **capacidade de identificar as melhores ferramentas para cada problema e justificar as escolhas**. Este roadmap foi estruturado com isso em mente: cada decisão tecnológica é acompanhada de (a) alternativas consideradas, (b) trade-offs avaliados, e (c) justificação concreta no contexto do TP2. Há ainda um apêndice final com frases prontas para o relatório e respostas defensivas para perguntas que possam surgir na apresentação.

## 1. Visão Geral

O TP2 evolui a infraestrutura monolítica do TP1 (sockets TCP + protocolo binário manual + SQLite) para uma arquitetura distribuída desacoplada, introduzindo três grandes alterações:

1. Substituição da comunicação direta Sensor→Gateway por um padrão Publish/Subscribe baseado em RabbitMQ.
2. Externalização de Pré-Processamento e Análise para serviços invocados via RPC (gRPC), em linguagem distinta (Python), demonstrando interoperabilidade.
3. Persistência em base de dados NoSQL (MongoDB) e interface de visualização para parametrização de análises.

## Diagrama lógico do sistema



|  
v  
[MongoDB] ---> [Interface CLI/Web]

## Princípios orientadores

- Desacoplamento — publisher não conhece subscriber; cliente RPC não conhece localização do servidor.
- Heterogeneidade — múltiplas linguagens e tecnologias colaboram via contratos bem definidos (Proto, JSON).
- Escalabilidade horizontal — múltiplos Gateways podem coexistir, cada um com responsabilidades distintas.
- Persistência e auditoria — dados e análises sobrevivem a reinício de qualquer componente.

## 2. Stack Tecnológica

A stack mantém continuidade com o TP1 (.NET para Sensores/Gateways/Servidor) e introduz Python para os serviços RPC e tecnologias distribuídas standard da indústria.

| Componente                | Tecnologia                                                                              |
|---------------------------|-----------------------------------------------------------------------------------------|
| <b>Sensor</b>             | .NET 8 (C#) — reutiliza simulador do TP1; usa RabbitMQ.Client para publicação.          |
| <b>Gateway</b>            | .NET 9 (C#) — subscritor RabbitMQ, cliente gRPC do Pré-Processamento.                   |
| <b>Servidor</b>           | .NET 8 (C#) — recetor de dados agregados, cliente gRPC da Análise, persiste em MongoDB. |
| <b>Pré-Processamento</b>  | Python 3.11 + grpcio — normalização de escalas/formatos.                                |
| <b>Análise / Previsão</b> | Python 3.11 + grpcio + pandas + scikit-learn — estatística e modelos.                   |
| <b>Pub/Sub</b>            | RabbitMQ 3.13 (Docker) — exchange topic com routing key hierárquica.                    |
| <b>BD</b>                 | MongoDB 7 (Docker) — coleções readings, analyses, sensors_metadata.                     |
| <b>Interface</b>          | CLI em .NET (Spectre.Console); Web em ASP.NET Core Razor Pages (opcional).              |
| <b>Containerização</b>    | Docker Compose — orquestra brokers, BD e serviços Python.                               |

## 3. Decisões Arquiteturais e Justificações

Esta secção documenta as cinco decisões tecnológicas estruturantes do projeto, comparando alternativas e justificando a escolha no contexto específico do TP2.

### 3.1 — RPC: gRPC vs alternativas

O protocolo não fixa a tecnologia de RPC. Foram avaliadas três alternativas:

| Tecnologia               | Pontos fortes                                                                   | Pontos fracos                                                         | Veredicto                                                                       |
|--------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------|
| gRPC                     | Multi-linguagem; HTTP/2; contratos .proto tipados; streams; ecossistema maduro. | Curva de aprendizagem do .proto; debug binário menos trivial.         | Escolhida — alinha-se com Python+.NET, contratos explícitos = boa nota técnica. |
| JSON-RPC over HTTP       | Simples; debug fácil; sem ferramentas extra.                                    | Sem tipagem forte; sem streams; performance inferior.                 | Descartada — demasiado próxima de uma API REST, perde valor demonstrativo.      |
| Java RMI / .NET Remoting | Integração nativa.                                                              | Mono-linguagem (Java↔Java ou .NET↔.NET); ferramentas em fim de vida.  | Descartada — bloqueia o uso de Python (valorização).                            |
| XML-RPC                  | Standard antigo; simples.                                                       | XML pesado; pouco uso atual.                                          | Descartada — anacrónica.                                                        |
| RabbitMQ RPC pattern     | Reaproveita o broker.                                                           | Mistura responsabilidades (broker vira síncrono); dificulta o ensino. | Descartada — confunde os dois padrões do protocolo.                             |

*"O gRPC foi escolhido por suportar nativamente as duas linguagens da nossa stack (C# e Python), por permitir a definição formal dos contratos em ficheiros .proto que tornam o protocolo de comunicação explícito e versionável, e pela eficiência do transporte HTTP/2 binário, adequado a fluxos de dados de telemetria."*

### 3.2 — Pub/Sub: estrutura de routing keys

O protocolo permite tópicos "por tipo de dado", "por zona" ou ambos. Foram avaliadas três modelações:

| Modelação               | Routing key            | Análise                                                                                                                                                                   |
|-------------------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Só por tipo             | pm25, temperatura, ... | Simples mas força cada Gateway a tratar todas as zonas de um tipo. Não permite especialização geográfica.                                                                 |
| Só por zona             | vilareal, porto, ...   | Cada Gateway gere uma zona mas mistura todos os tipos. Não permite especialização funcional.                                                                              |
| Hierárquica (escolhida) | zona.tipo.idSensor     | Aproveita o topic exchange com wildcards (*, #). Permite subscrição por zona (vilareal.#), por tipo (*.pm25.#), ou combinação. Cumpre literalmente o "e/ou" do enunciado. |

#### Exemplo de bindings com a Opção C

| Gateway     | Binding            | Subscreve                                 |
|-------------|--------------------|-------------------------------------------|
| GW-VilaReal | vilareal.#         | Tudo da zona de Vila Real                 |
| GW-Poluição | *.pm25.# + *.no2.# | PM2.5 e NO <sub>2</sub> de todas as zonas |
| GW-Porto-Ar | porto.pm25.#       | Só PM2.5 do Porto                         |
| GW-Master   | #                  | Tudo (auditoria/debug)                    |

*"Optou-se por uma routing key hierárquica zona.tipo.idSensor sobre um topic exchange do RabbitMQ. Esta solução evidencia o desacoplamento Sensor↔Gateway (os Sensores publicam sem conhecer os Gateways) e aproveita os wildcards \* e # para permitir múltiplos perfis de Gateway sem alterações no código dos publishers — bastando reconfigurar bindings."*

### 3.3 — Base de Dados: MongoDB vs PostgreSQL

O protocolo permite "relacional ou NoSQL". Foram avaliadas as duas alternativas dominantes:

| Critério                 | MongoDB (escolhida)                                                     | PostgreSQL                                                                         |
|--------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Heterogeneidade de dados | Documentos com schemas flexíveis — cada tipo de sensor tem o seu shape. | Tabela genérica (EAV) ou múltiplas tabelas — perde tipagem ou multiplica esquemas. |
| Ingestão write-intensive | Otimizado para writes; sharding nativo.                                 | Boa, mas requer mais cuidado com índices/locking.                                  |
| Fluxo de dados           | JSON-nativo — alinha com RabbitMQ + gRPC + interface.                   | Requer parsing/mapping ORM em cada step.                                           |
| Time-series              | Coleções time-series nativas (Mongo 5+).                                | Requer extensão TimescaleDB.                                                       |
| Queries típicas          | Filtros por sensor+intervalo: índice {sensorId:1, ts:-1} + agregações.  | WINDOW functions poderosas, mas overhead para casos simples.                       |
| Relações fortes          | Pouco eficiente — mas o domínio quase não tem.                          | Forte — vantagem não aplicável aqui.                                               |

*"Optou-se por MongoDB pela natureza heterogénea dos dados produzidos pelos sensores, pela ingestão write-intensive característica de cenários IoT, e pelo alinhamento com o fluxo JSON já existente entre RabbitMQ, gRPC e a interface. A ausência de relações fortes no domínio elimina a vantagem clássica das bases relacionais."*

### 3.4 — Linguagem dos serviços RPC: Python

O protocolo valoriza heterogeneidade tecnológica. A escolha entre manter tudo em .NET ou introduzir Python foi avaliada assim:

| Razão            | Justificação                                                                                                                                 |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Análise de dados | Python tem o ecossistema dominante (pandas, numpy, scikit-learn, statsmodels). Implementar regressão/outliers em C# seria reinventar a roda. |

| Razão                              | Justificação                                                                                                     |
|------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Heterogeneidade                    | O protocolo refere explicitamente a inclusão de "diferentes tecnologias e linguagens" como fator de valorização. |
| Demonstração de interoperabilidade | gRPC entre C# e Python valida o argumento dos contratos .proto cross-language.                                   |
| Comunidade                         | Tutoriais e exemplos académicos abundantes para gRPC+Python.                                                     |
| Pré-Proc + Análise em Python       | Decisão tomada por consistência: ambos os serviços partilham bibliotecas e padrão; mais fácil de manter.         |

### 3.5 — Orquestração: Docker Compose

A montagem manual de RabbitMQ + MongoDB + dois serviços Python é frágil e dificulta a reprodução. Docker Compose foi escolhido por:

- Um único comando (docker compose up) arranca toda a infraestrutura — facilita avaliação.
- Versões fixas (rabbitmq:3.13, mongo:7) — elimina "works on my machine".
- Healthchecks asseguram ordem de arranque (Servidor só inicia após Mongo estar pronto).
- Volumes nomeados garantem persistência entre restarts.
- É o padrão de mercado para protótipos distribuídos — boa demonstração técnica.

### 3.6 — Decisões Operacionais e de Âmbito

Para além das decisões tecnológicas estruturantes (3.1 a 3.5), o grupo tomou — após reflexão e discussão informada com o docente — um conjunto de decisões operacionais que delimitam o âmbito do projeto. O docente indicou explicitamente que estas decisões devem ser tomadas pelo grupo, justificadas, e defendidas na apresentação.

| Tema                          | Decisão                                                                                                                                                | Justificação para defender                                                                                                                                                                                             |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reutilização do TP1           | Refatoração mínima: preservar simulação e agregação; eliminar sockets diretos; substituir apenas os pontos de comunicação.                             | O protocolo do TP2 exige "manter funcionalidades anteriores". Reduz risco e preserva trabalho já validado. Permite focar esforço no que é novo (Pub/Sub, RPC, BD).                                                     |
| Profundidade da Análise       | Estatística descritiva (média, desvio, mediana, percentis) + outliers (z-score) + tendência (regressão linear) + previsão (EWMA). Sem redes neuronais. | Cobre os exemplos explícitos do enunciado ("estatística, deteção de padrões, previsão de riscos"). ML profundo distrairia do âmbito SD; demonstra mesmo assim uso do ecossistema Python (pandas, numpy, scikit-learn). |
| Formatos no Pré-Processamento | Suportar os três formatos referidos no enunciado: JSON, XML e CSV. Cada Sensor pode publicar num formato distinto.                                     | O protocolo refere-os explicitamente. Demonstrar os 3 prova generalidade do mecanismo de normalização — diferencia do TP1 que só usava formato binário fixo.                                                           |
| Interface                     | CLI (Spectre.Console) obrigatória e completa; Web (ASP.NET Razor) opcional como valorização caso o cronograma permita.                                 | CLI é mais demo-friendly em apresentação ao vivo (headless, scriptável, sem dependência de browser). Prioriza-se uma CLI funcional sobre uma Web meia-construída.                                                      |

| Tema                       | Decisão                                                                                                                                                                       | Justificação para defender                                                                                                                                                                                        |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Multi-linguagem            | .NET (Sensores/Gateways/Servidor) + Python (Pré-Proc e Análise) + 1 Sensor em Node.js como extra de valorização.                                                              | Duas linguagens já cumprem o requisito do enunciado; um único Sensor em Node.js (com amqplib) acrescenta uma terceira a custo baixo e demonstra interoperabilidade real entre 3 stacks distintas no mesmo broker. |
| Grupo (3 membros)          | Divisão por fase: M-A=Sensores+RabbitMQ+Pub/Sub; M-B=Serviços Python+gRPC; M-C=Servidor+MongoDB+CLI. Docker/relatório/testes partilhados.                                     | Paraleliza trabalho ( $\approx 1$ semana/fase mas em paralelo); cada membro domina uma fase e apresenta-a; pair-review cruzado entre membros garante qualidade e conhecimento partilhado.                         |
| Apresentação               | $\approx 15$ minutos: $\leq 5$ slides (arquitetura, decisões com tabelas comparativas, lições aprendidas) + demo ao vivo ( $\approx 5$ min) + Q&A. Todos os membros intervêm. | Formato típico de aula PL. Foco máximo em demo e justificações — que é o critério avaliado. Slides curtos evitam leitura passiva e libertam tempo para Q&A.                                                       |
| Estratégia de avaliação    | Quatro frentes paralelas: relatório com tabelas comparativas + código limpo e comentado + demo robusta com docker compose up + Q&A defensivo preparado (ver Apêndice B).      | Distribui o esforço pelos vetores avaliados; nenhum fica fraco. A capacidade de defender as escolhas é cobrada explicitamente pelo docente.                                                                       |
| Escalabilidade da demo     | Demo ao vivo: 2–3 Gateways + 4–6 Sensores em paralelo. Teste de carga separado: simulação de 100 sensores.                                                                    | Múltiplos componentes em paralelo provam escalabilidade real (não só teórica) e validam a routing key hierárquica (3.2). 100 sensores no teste de carga validam throughput sem encher o ecrã na demo.             |
| Persistência e resiliência | Mensagens RabbitMQ durable + ack manual + AutomaticRecoveryEnabled no .NET client + retry com Polly no gRPC.                                                                  | One Health é monitorização contínua — perder leituras é inaceitável. Custo de implementação mínimo, ganho grande em narrativa de robustez para o relatório.                                                       |
| Segurança                  | Credenciais via variáveis de ambiente; utilizador dedicado em RabbitMQ e MongoDB (não usar guest/root). Sem TLS no MVP.                                                       | Boa prática mínima sem complexidade. TLS exigiria gestão de certificados e CA — desproporcional ao âmbito do TP2, que não menciona segurança como requisito.                                                      |

## 4. Faseamento do Trabalho

O desenvolvimento segue as 3 fases propostas no protocolo, com sub-tarefas, critérios de aceitação claros, e — para cada fase — uma subsecção de "Alternativas consideradas" que reforça a defesa das escolhas perante o docente.

## FASE 1 — Implementação das chamadas RPC (gRPC)

*Duração estimada: ≈ 1 semana*

Objetivo: implementar comunicação RPC entre Gateway↔Pré-Proc e Servidor↔Análise, mantendo temporariamente a comunicação por sockets do TP1 entre Sensor↔Gateway. Permite validar o RPC isoladamente.

### Tarefa 1.1 — Definir contratos .proto

- Criar diretoria proto/ na raiz do repositório (partilhada entre projetos).
- preprocessing.proto: rpc Normalize(RawReading) returns (NormalizedReading); inclui sensorId, type, value, unit, timestamp, rawFormat (JSON/XML/CSV).
- analysis.proto: rpc Analyze(AnalysisRequest) returns (AnalysisResult); rpc Predict(PredictionRequest) returns (PredictionResult); requests parametrizáveis por janela temporal, tipo, sensor.
- Gerar stubs C# via Grpc.Tools no .csproj; stubs Python via python -m grpc\_tools.protoc.
- Versionar os .proto desde o início — qualquer alteração afeta ambos os lados.

### Tarefa 1.2 — Serviço de Pré-Processamento (Python)

- Criar services/preprocessing/ com servidor gRPC.
- Implementar conversão de escalas (Fahrenheit→Celsius, Pa→hPa).
- Parsing de JSON, XML (xml.etree) e CSV (csv stdlib) — demonstrar os 3 formatos do enunciado.
- Validação de ranges (descartar leituras impossíveis: PM2.5 negativo, temperatura > 80°C).
- Porta 50051. Dockerfile com Python 3.11-slim. Healthcheck.

### Tarefa 1.3 — Serviço de Análise (Python)

- services/analysis/ com servidor gRPC, biblioteca pandas + numpy.
- Análises base: média, desvio padrão, mediana, percentis sobre janela temporal.
- Detecção de outliers via z-score (limiar configurável).
- Análise de tendência (slope de regressão linear simples).
- Previsão simples: extrapolação linear ou média móvel exponencial (EWMA) sobre N pontos futuros.
- Porta 50052. Dockerfile, healthcheck.

### Tarefa 1.4 — Cliente gRPC no Gateway (C#)

- Adicionar Grpc.Net.Client, Google.Protobuf, Grpc.Tools ao Gateway.csproj.
- Configurar canal persistente (GrpcChannel.ForAddress("http://preprocessing:50051")) — reutilizável.
- Antes de agregar/encaminhar, chamar Normalize() para cada leitura.
- Polly para retry exponencial em caso de falha do serviço.

### Tarefa 1.5 — Cliente gRPC no Servidor (C#)

- Expor endpoint local (CLI ou API) que recebe pedidos de análise parametrizada.
- Invocar AnalysisService.Analyze() com janela temporal, tipo, sensor.
- Para já, armazenar resultado em memória (MongoDB chega na Fase 3).

### Alternativas consideradas e descartadas

- REST/JSON-RPC em vez de gRPC — descartado por não ter contratos fortes e perder valor demonstrativo (já visto em outras disciplinas).
- Comunicação síncrona pelo próprio RabbitMQ (RPC pattern) — descartado por confundir os dois padrões e dificultar a separação conceptual no relatório.
- Tudo em .NET (sem Python) — descartado por não aproveitar a valorização por heterogeneidade nem o ecossistema de análise.

#### Critérios de aceitação — Fase 1

- Stubs gerados a partir dos .proto compilam em ambas as linguagens sem warnings.
- É possível iniciar serviços Python (docker compose up preprocessing analysis) e clientes C# invocam funções remotas com sucesso.
- Logs cross-language coerentes (sensorId aparece nas duas pontas).
- Latência típica < 50ms em localhost para uma chamada de Normalize().

## FASE 2 — Comunicação Pub/Sub entre Sensores e Gateways

*Duração estimada: ≈ 1 semana*

Objetivo: substituir os sockets diretos Sensor↔Gateway por mensagens via RabbitMQ usando a estrutura hierárquica decidida em 3.2.

#### Tarefa 2.1 — Configurar RabbitMQ

- Adicionar rabbitmq:3.13-management ao docker-compose.yml (portas 5672 e 15672).
- definitions.json idempotente: cria utilizador, vhost, exchange sensors.exchange (type=topic, durable=true).
- Validar painel web em http://localhost:15672.

#### Tarefa 2.2 — Modelo de tópicos (decisão em 3.2)

- Exchange: sensors.exchange (type: topic, durable: true)
- Routing key: <zona>.<tipo>.<idSensor> (ex: vilareal.pm25.s07)
- Mensagem (JSON): { sensorId, zone, type, value, unit, timestamp, rawFormat }
- Delivery mode: 2 (persistente)

#### Tarefa 2.3 — Publisher no Sensor (.NET)

- Adicionar RabbitMQ.Client ao Sensor.csproj.
- Remover socket TCP do TP1. Substituir por BasicPublish() no sensors.exchange.
- Configurar via appsettings.json: host, porta, credenciais, zona, tipoSensor, idSensor, intervaloPublicacao.
- Reconexão automática com backoff exponencial (AutomaticRecoveryEnabled = true do .NET client).
- Suportar diferentes formatos de payload (JSON/XML/CSV) — campo rawFormat indica o formato bruto antes do Pré-Proc.

#### Tarefa 2.4 — Subscriber no Gateway (.NET)

- Remover servidor TCP. Adicionar AsyncEventingBasicConsumer.
- Configurar bindings por zona/tipo via appsettings.json (lista de routing patterns).
- Para cada mensagem: invocar pré-processamento (gRPC) → agregar → encaminhar para Servidor.



- Ack manual após sucesso; nack + requeue em caso de erro transitório; descartar (sem requeue) em erro permanente.

### Tarefa 2.5 — Múltiplos Gateways

- Demonstrar arranque de  $\geq 2$  Gateways com bindings distintos (ex: GW-VilaReal vs GW-Poluição).
- Validar pela visualização das queues no painel RabbitMQ (mensagens distribuídas conforme bindings).

### Alternativas consideradas e descartadas

- Apache Kafka — overkill para o âmbito do TP2; arranque mais pesado; menos didático para Pub/Sub clássico.
- Redis Pub/Sub — não tem mensagens persistentes (fire-and-forget); inadequado para auditoria/recuperação.
- MQTT — apropriado para IoT-puro mas distancia-se do âmbito "distribuído urbano" do protocolo, que pede RabbitMQ explicitamente.
- Direct exchange — perderia a flexibilidade dos wildcards; force-fit do modelo "só por tipo" ou "só por zona".

### Critérios de aceitação — Fase 2

- Os sensores publicam sem conhecer os gateways (zero referências a IPs/portos de gateways).
- Adicionar/remover Gateways em runtime não exige alterar sensores.
- Mensagens persistentes (sobrevivem a restart do broker) — validar parando e arrancando o container.
- Painel RabbitMQ mostra exchange, queues e taxa de throughput coerentes.

## FASE 3 — Funcionalidades Adicionais — BD + Interface

*Duração estimada:  $\approx 1$  semana*

Objetivo: persistir leituras agregadas e resultados de análises em MongoDB; oferecer interface para consulta e despoletamento de novas análises.

### Tarefa 3.1 — Configurar MongoDB

- mongo:7 no docker-compose.yml (porta 27017). Volume nomeado mongo\_data.
- Coleções: readings (leituras agregadas), analyses (resultados), sensors\_metadata.
- Índices: {sensorId: 1, timestamp: -1} em readings; {type: 1, createdAt: -1} em analyses.
- Avaliar uso de coleção time-series para readings (Mongo 5+).

### Tarefa 3.2 — Camada de acesso a dados (.NET)

- Adicionar MongoDB.Driver ao Servidor.csproj.
- Repositories/{ReadingsRepository,AnalysesRepository}.cs com operações CRUD assíncronas.
- Definir POCOs com [BsonElement] para mapeamento limpo (sem Document genérico).
- Connection string em appsettings.json; user/password via variáveis de ambiente.

### Tarefa 3.3 — Persistência no fluxo principal

- No Servidor: gravar cada batch agregado recebido dos Gateways em readings.
- Após cada análise (via gRPC), gravar resultado em analyses com referência aos parâmetros.

- Validar consistência: o que se lê via interface é exatamente o que foi publicado pelos sensores (rastreadabilidade).

#### Tarefa 3.4 — Interface CLI (Spectre.Console)

- Comando `list-readings --sensor X --from Y --to Z` — tabela colorida com filtros.
- Comando `analyze --type pm25 --zone vilareal --from Y --to Z` — dispara análise via gRPC, mostra resultado e guarda.
- Comando `list-analyses` — histórico de análises com IDs e parâmetros.
- Comando `show-sensor --id X` — mostra metadados e últimas N leituras.

#### Tarefa 3.5 — Interface Web (opcional, valorização)

- ASP.NET Core Razor Pages — porta 5000.
- Página `/readings` com filtros e paginação; `/analyses`; `/new-analysis` com formulário.
- Gráficos com Chart.js para séries temporais.

#### Alternativas consideradas e descartadas

- PostgreSQL — descartado pela análise em 3.3 (heterogeneidade, write-intensive, alinhamento JSON).
- SQLite (continuidade com TP1) — descartado por não ser servidor de BD (single-file não encaixa em sistema distribuído).
- Interface só Web (sem CLI) — descartado por CLI ser mais demonstrativa em apresentação ao vivo e mais simples para demo headless.

#### Critérios de aceitação — Fase 3

- Dados sobrevivem a restart de todos os componentes (verificar via `docker compose down && up`).
- Consultas com filtros temporais devolvem em < 1s para 100k leituras.
- Interface permite parametrizar e visualizar análises sem editar código.

## 5. Tarefas Transversais

Estas tarefas correm em paralelo com as três fases e são fundamentais para a qualidade da entrega.

### 5.1 — Repositório e gestão de issues

- Criar repositório no GitHub/GitLab no início do projeto.
- Issue por sub-tarefa do roadmap; labels (fase-1, fase-2, fase-3, docs, infra).
- Branch por feature; PR com revisão entre membros (pair-review) antes de merge para main.
- README.md com instruções de arranque (`docker compose up`) e arquitetura.

#### Divisão de tarefas pelos 3 membros

| Membro | Responsabilidade principal    | Detalhes                                                                                                                                                    |
|--------|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| M-A    | Sensores + RabbitMQ + Pub/Sub | Refatorar Sensor (.NET) para publisher; 1 Sensor em Node.js (extra); configurar exchange/queues/bindings; suporte aos 3 formatos (JSON/XML/CSV) no payload. |

| Membro | Responsabilidade principal         | Detalhes                                                                                                                                          |
|--------|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| M-B    | Serviços Python + gRPC             | Definir .proto; implementar Pré-Processamento (normalização, parsing); implementar Análise (estatística, outliers, regressão, EWMA); Dockerfiles. |
| M-C    | Servidor + MongoDB + Interface CLI | Refatorar Servidor (.NET) para subscriber + cliente gRPC; repositories MongoDB; CLI Spectre.Console; Web (se houver tempo).                       |
| Todos  | Transversal                        | docker-compose.yml; relatório técnico (cada um escreve a secção da sua fase); testes de integração; preparação da demo e Q&A.                     |

## 5.2 — Configuração e arranque (Docker Compose)

- Um único docker-compose.yml: rabbitmq, mongo, preprocessing, analysis.
- .env.example versionado; .env real ignorado pelo Git.
- Healthchecks; depends\_on com condition: service\_healthy.

## 5.3 — Testes

- Unitários nos serviços Python (pytest) — normalização, análise.
- Integração ponta-a-ponta: docker compose up → publicar X mensagens → validar aparecem em MongoDB.
- Script de carga: N sensores em paralelo; medir throughput e perdas.

## 5.4 — Relatório técnico (4 páginas máximo)

- Secção 1 — Protocolo: JSON schema das mensagens Pub/Sub, .proto, fluxos.
- Secção 2 — Implementação: aproveitar as justificações desta secção 3 deste roadmap.
- Anexo — link para repositório + screenshots das issues.
- Escrever em paralelo (não deixar para a véspera).

## 5.5 — Apresentação

- Demo ao vivo: arranque com docker compose, geração de dados, painel RabbitMQ, análise via CLI.
- Slides curtos (≤10): arquitetura, decisões (com tabelas comparativas), demo, lições.

# 6. Cronograma Sugerido

Calendário proposto considerando a data limite de 29 de maio de 2026:

| Período   | Foco   | Entregáveis                                                                             |
|-----------|--------|-----------------------------------------------------------------------------------------|
| 15–17 mai | Setup  | Repositório, docker-compose esqueleto, .proto definidos, esboço do relatório.           |
| 18–20 mai | Fase 1 | Serviços Python (Pré-Proc + Análise) operacionais; clientes gRPC em Gateway e Servidor. |

| Período   | Foco    | Entregáveis                                                            |
|-----------|---------|------------------------------------------------------------------------|
| 21–23 mai | Fase 2  | RabbitMQ + Sensor publisher + Gateway subscriber + múltiplos Gateways. |
| 24–26 mai | Fase 3  | MongoDB + repositórios + CLI; Web (se houver tempo).                   |
| 27–28 mai | Polish  | Testes integração, screenshots, fechar issues, finalizar relatório.    |
| 29 mai    | Entrega | Submissão Moodle + preparação da apresentação.                         |

## 7. Riscos e Mitigações

| Risco                          | Mitigação                                                                               |
|--------------------------------|-----------------------------------------------------------------------------------------|
| Curva de gRPC + Python         | Começar Fase 1 cedo; tutorial oficial; manter contratos simples; gerar stubs no início. |
| Conflitos no driver MongoDB    | Fixar versão MongoDB.Driver no .csproj; testar com Mongo 7.                             |
| RabbitMQ — mensagens perdidas  | Painel web para diagnóstico; mensagens persistentes; ack manual.                        |
| Falta de tempo para Web UI     | CLI cumpre requisitos; Web é valorização.                                               |
| Conflitos no Git entre membros | Branches feature/fase-X-tarefa-Y; PRs revisados; merges frequentes.                     |
| Relatório deixado para o fim   | Atualizar secções no fecho de cada tarefa.                                              |
| Docker no Windows lento        | Usar WSL2; configurar memória mínima 4GB.                                               |

## Apêndice A — Frases prontas para o relatório

Excertos curtos para integrar nas secções de Implementação e Justificação do relatório (4 páginas).

### Sobre gRPC

*O gRPC foi escolhido por suportar nativamente as duas linguagens da nossa stack (C# e Python), por permitir a definição formal dos contratos em ficheiros .proto que tornam o protocolo de comunicação explícito e versionável, e pela eficiência do transporte HTTP/2 binário, adequado a fluxos de telemetria. Alternativas como JSON-RPC ou XML-RPC foram descartadas pela ausência de tipagem forte e pelo menor valor demonstrativo no contexto de Sistemas Distribuídos.*

### Sobre o RabbitMQ e routing hierárquica

*Adotou-se uma routing key hierárquica do tipo zona.tipo.idSensor sobre um topic exchange do RabbitMQ. Esta estrutura permite que cada Gateway subscreva por zona geográfica, por tipo de dado, ou pela combinação de ambos, usando wildcards (\*, #). O resultado é o desacoplamento total Sensor↔Gateway: os Sensores publicam sem conhecer os destinatários, e a topologia de subscrição pode ser reconfigurada em runtime sem alterar publishers — uma propriedade nuclear de sistemas distribuídos escaláveis.*

### Sobre o MongoDB

*Optou-se por MongoDB pela natureza heterogênea dos dados produzidos pelos sensores (cada tipo tem metadados próprios), pela ingestão write-intensive característica de cenários IoT, pelo alinhamento com o fluxo JSON já existente entre RabbitMQ, gRPC e a interface, e pela existência de coleções time-series nativas. A ausência de relações fortes no domínio elimina a vantagem clássica das bases relacionais.*

### Sobre Python

*Os serviços de Pré-Processamento e Análise foram implementados em Python para aproveitar o ecossistema dominante de análise de dados (pandas, numpy, scikit-learn) e para demonstrar interoperabilidade real entre linguagens via contratos gRPC. Esta heterogeneidade tecnológica é também um dos fatores de valorização explicitamente referidos no enunciado do TP2.*

### Sobre Docker Compose

*Toda a infraestrutura (broker RabbitMQ, base de dados MongoDB, serviços Python) é orquestrada por Docker Compose, garantindo reprodutibilidade, isolamento de versões e arranque em um único comando. Esta é a abordagem standard da indústria para sistemas distribuídos em ambiente de desenvolvimento.*

## Apêndice B — Perguntas defensivas (Q&A)

Possíveis perguntas do docente na apresentação e respostas a preparar.

| Pergunta                                                        | Resposta a preparar                                                                                                                                                                                                                              |
|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Porque não usaram gRPC também entre Sensor e Gateway?           | Porque o cenário Sensor→Gateway é de muitos publishers para muitos subscribers, sem necessidade de resposta — encaixa em Pub/Sub. RPC é apropriado quando há um par cliente-servidor com resposta esperada (Gateway→Pré-Proc, Servidor→Análise). |
| E se o RabbitMQ falhar? O sistema para?                         | Os Sensores têm reconexão automática com backoff; o broker em produção seria clusterizado. Para o âmbito do TP2, a persistência das mensagens garante que nada se perde durante o downtime; após restart, o backlog é processado.                |
| Porque routing hierarquica e não fanout para todos os Gateways? | Fanout entregaria a todos os Gateways todas as mensagens - desperdício de banda e processamento. Topic com wildcards filtra na origem, permitindo Gateways especializados.                                                                       |
| Como garantem que não se perde uma leitura?                     | Quatro mecanismos: (1) mensagem persistente; (2) ack manual após processamento no Gateway; (3) gravação atômica em MongoDB; (4) retry no cliente gRPC se Pré-Proc falhar.                                                                        |
| E a escalabilidade? Aguenta 1000 sensores?                      | RabbitMQ aguenta centenas de milhares de msg/s; podemos escalar horizontalmente Gateways subscvendo as mesmas queues (round-robin). Bottleneck seria o Servidor - resolvido com sharding por zona.                                               |
| Porque não usaram um ORM como Entity Framework?                 | MongoDB.Driver já oferece mapeamento POCO via atributos; um ORM adicional acrescentaria complexidade sem benefício para o domínio simples (sem joins, sem migrations).                                                                           |

| Pergunta                                                   | Resposta a preparar                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Como testaram o sistema?                                   | Testes unitarios em Python (pytest); script de integracao que publica N mensagens e verifica que aparecem em MongoDB; teste de carga com simulacao de 100 sensores em paralelo.                                                                                                                       |
| Porque mantiveram codigo do TP1 em vez de reescrever?      | Refatoracao minima preserva trabalho ja validado e cumpre o requisito explicito do enunciado de manter funcionalidades anteriores. Eliminamos apenas os sockets diretos, substituindo pelos pontos de comunicacao Pub/Sub e RPC.                                                                      |
| Porque tres formatos no Pre-Processamento? Nao bastava um? | O enunciado refere explicitamente JSON, XML e CSV. Suportar os tres prova que o mecanismo de normalizacao e generalizavel - nao esta acoplado a um formato especifico. Cada Sensor publica no formato mais natural a sua origem.                                                                      |
| Porque a interface e CLI e nao Web?                        | CLI e mais robusta para demo ao vivo (headless, scriptavel, sem dependencia de browser) e suficiente para parametrizar todas as analises exigidas. Web foi considerada como valorizacao, implementada apenas se o cronograma o permitiu.                                                              |
| Porque incluíram um Sensor em Node.js?                     | O enunciado valoriza explicitamente a inclusao de diferentes tecnologias e linguagens. Ja temos .NET + Python; um Sensor em Node.js com amqplib acrescenta uma terceira stack a custo muito baixo (cerca de 30 linhas) e demonstra interoperabilidade real entre 3 linguagens distintas via RabbitMQ. |
| Porque nao implementaram TLS / autenticao forte?           | O ambito do TP2 nao menciona seguranca como requisito; implementar TLS exigiria gestao de certificados e CA, desproporcional ao MVP. Aplicamos boa pratica minima: credenciais via variaveis de ambiente e utilizadores dedicados (nao guest/root) no RabbitMQ e MongoDB.                             |
| E se quisessem escalar para uma cidade inteira?            | A arquitetura escala horizontalmente: (1) multiplos Gateways consumindo as mesmas queues distribuem carga em round-robin; (2) Servidor pode ser sharded por zona; (3) MongoDB suporta sharding nativo; (4) RabbitMQ pode ser clusterizado. A demo prova o principio com 100 sensores.                 |

## Apendice C - Checklist final (antes de submeter)

- Repositorio com README detalhado e instrucoes de arranque.
- docker compose up arranca todos os servicos sem erros.
- Sensores publicam e Gateways recebem via RabbitMQ (validado no painel).
- Gateways invocam Pre-Processamento (Python) com sucesso via gRPC.
- Servidor invoca Analise (Python) e armazena resultados em MongoDB.
- Interface CLI permite consultar dados e disparar analises parametrizadas.
- Issues criadas e fechadas correspondem as tarefas reais executadas.
- Relatorio tecnico entregue em PDF com protocolo, implementacao, justificacoes e anexo.

- Apresentacao preparada com demo ao vivo + slides com tabelas comparativas.
- Q&A defensivo (Apendice B) revisto pelos 3 membros.